CPD - Project 1

Performance evaluation of a single core and a multi-core implementation

Realizado por:
José Ribeiro - up202108868
Rita Leite - up202105309
Tiago Azevedo - up202108699

Índice

1. Problem Description **3**
2. Explanation of the Algorithms **3**
3. Performance Metrics **3**
4. Results and Analysis **3**
4.1. Differences in Normal and Line Multiplication's Algorithms 4
4.2. Differences in Block Multiplication's Algorithm 5
4.3. Differences in Parallel Implementations 6
5. Conclusion **8**
Attachments **9**
A1 - Normal Multiplication's Algorithm Results 9
A2 - Line Multiplication's Algorithm Results 10
A3 - Block Multiplication's Algorithm Results 11
A4 - First Parallel Algorithm Results 12

1. Problem Description

This project was carried out within the scope of the CPD discipline, of the Computer Engineering course at FEUP. Its goal was the study of the effect of memory hierarchy and of different access patterns to an array on processor performance when accessing large amounts of data. To this end, we applied different algorithms to calculate the product of two matrices, also making use of the performance API (PAPI) to collect relevant performance indicators from the program execution.

2. Explanation of the Algorithms

We started by implementing three sequential algorithms. The first one is the **normal multiplication** algorithm where the result is obtained by multiplying one line of the first matrix by each column of the second matrix, which is the most common method for multiplying two matrices. To the second implementation we call the **line multiplication** algorithm because it starts by multiplying each element of the first matrix by the corresponding row in the second matrix, before moving on to the next element. The last one is the **block multiplication**, where we start by dividing the matrices into smaller ones with a specific size, and then, for each one of them, we run the same algorithm as used in the line multiplication.

We also made alterations to the line multiplication algorithm so that we could make use of parallel computation. The **first version** parallelizes the outermost loop (loop over 'i'), so each thread works on a separate chunk of the outer loop iterations with size n/p where n is the matrix size and p is the number of threads (in our case, 8), which means that each thread executes a subset of iterations of 'i', where for each 'i', the inner loops ('k' and 'j') are executed sequentially within each thread. The **second version** parallelizes the innermost loop (loop over 'j'). This means that all iterations of the 'i' and 'k' loops will be executed by all threads in the parallel region, but the 'j' loop iterations will be distributed among the threads.

3. Performance Metrics

Through the use of the performance API (PAPI), we were able to store the number of cache misses to the L1 and L2 cache. In addition, we also recorded the execution time for each matrix size and algorithm, which also allowed us to calculate the FLOPS.

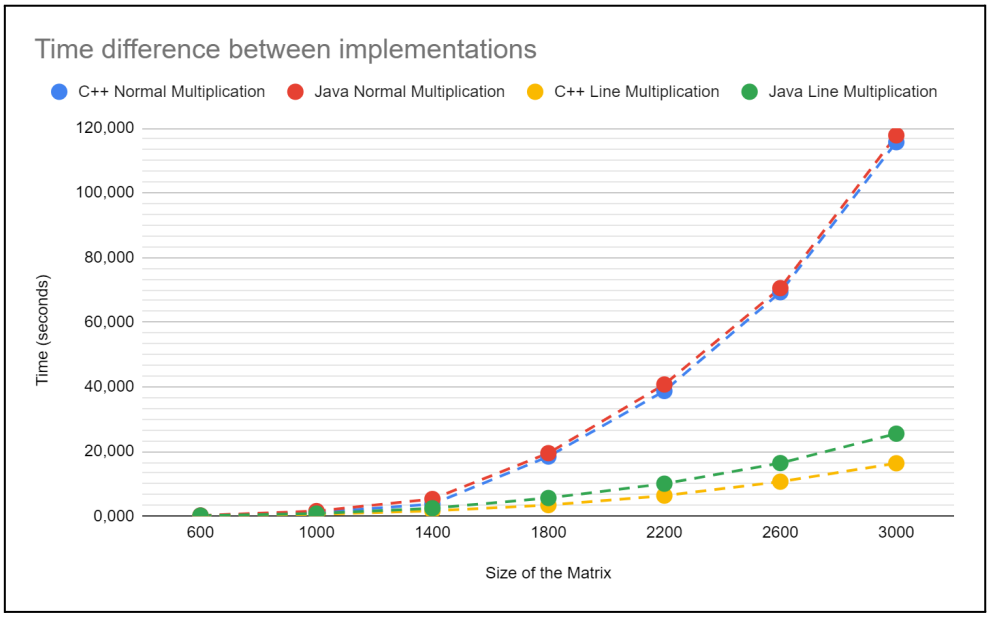
In the cases where parallel computing was used, we also calculated the speedup and efficiency for each case.

4. Results and Analysis

For each algorithm and matrix' size, we run the code ten times. Due to this, the results presented in the attachments come from the calculation of the average time and the average values of L1 Data Cache Misses and L2 Data Cache Misses. The GFlops were calculated using the average time.

4.1. Differences in Normal and Line Multiplication's Algorithms

The Normal Multiplication algorithm grows exponentially as the size of the matrices increases. However, it is possible to see that the Line Multiplication algorithm grows in a much more constant way, where it is possible to notice a greater difference between the results in C++ and Java. We registered 10 values for each case and we noticed that the first times obtained in Java, for each matrix size, were considerably smaller than the following ones. For example, in the case of the 3000x3000 matrix, the first result obtained was 17,115 seconds, but the following results were substantially higher, with the average being 25,528 seconds.



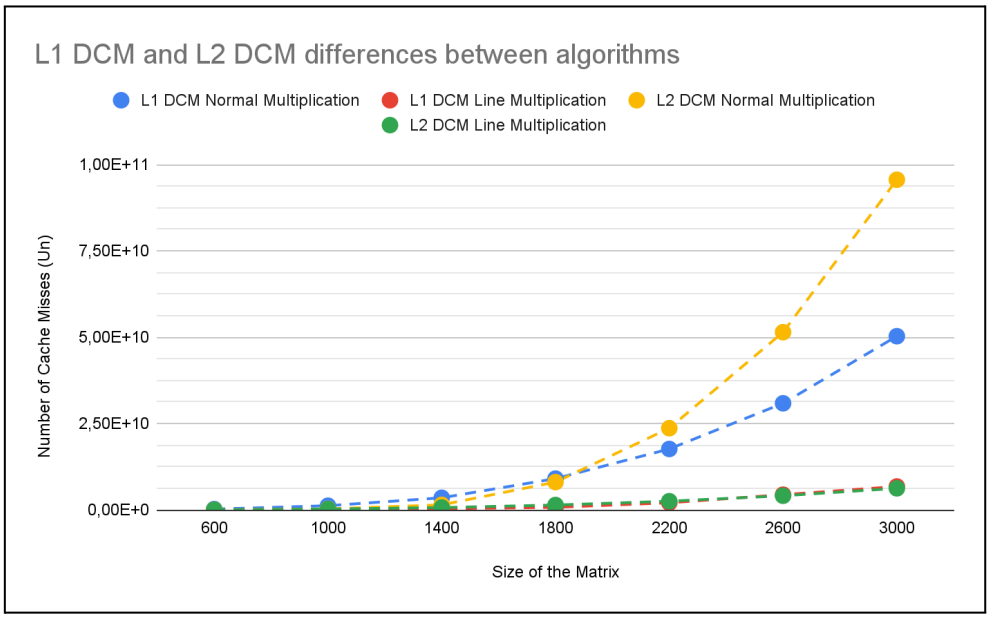
The Line Multiplication algorithm is much faster and more efficient than the Normal Multiplication algorithm, being even more noticeable as the size of the matrix increases. This difference happens due to the way the cache is accessed throughout the process, and we can see the differences between the number of misses in the L1 and L2 caches.

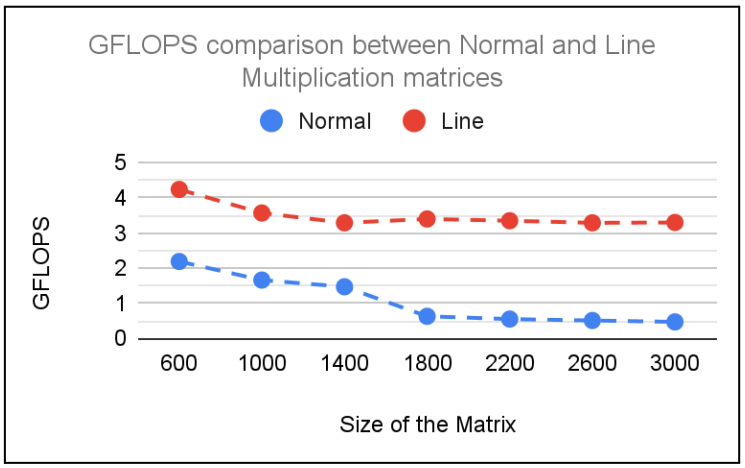
First of all, it is important to keep in mind that whenever a value is needed, a copy is made to the cache of an entire block of memory that contains that value.

In the Normal Multiplication algorithm, it starts in the first loop by accessing a block of memory containing the row of the first matrix that will be multiplied, but then it will fetch the first element of each column of the second matrix. This method becomes inefficient as the elements of the matrix that are in the same column may be very far from each other in memory (number of columns), meaning that they may not be found in the block copied to the cache. However, in the Line Multiplication algorithm, since it is row by row, the problem that occurred before when doing it by columns does not occur. Since blocks of memory with continuous values are copied, they will correspond to a line, or part of it, that is, they can already be in the cache when they are needed.

We can also observe that the number of cache misses in the normal algorithm increases exponentially with the increase in matrix dimensions, as the memory distance of the values used increases. This results in a lower probability of these values already being in the lower levels of memory, making it necessary to access higher levels more frequently to

fetch the data, and, consequently, for the same number of operations, a lower value of FLOPS.

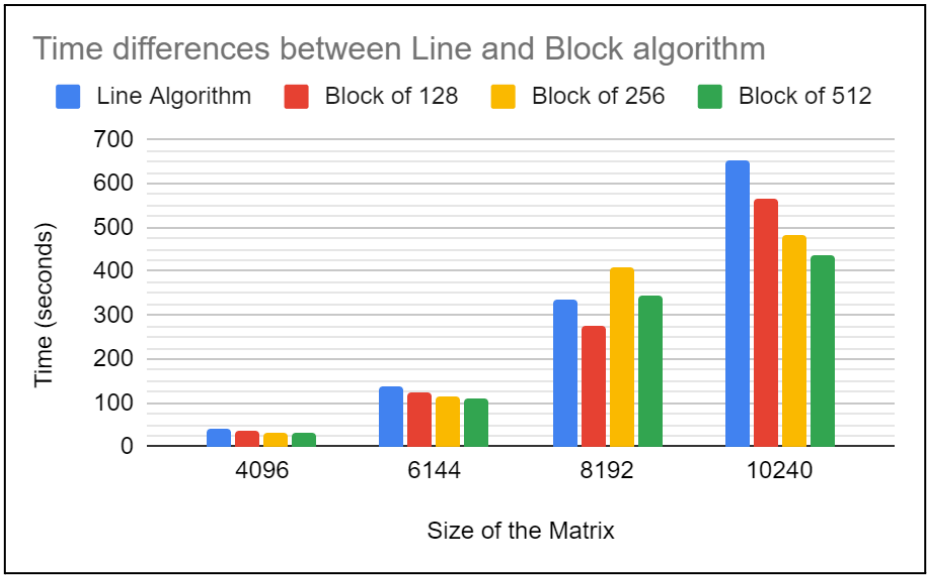




4.2. Differences in Block Multiplication's Algorithm

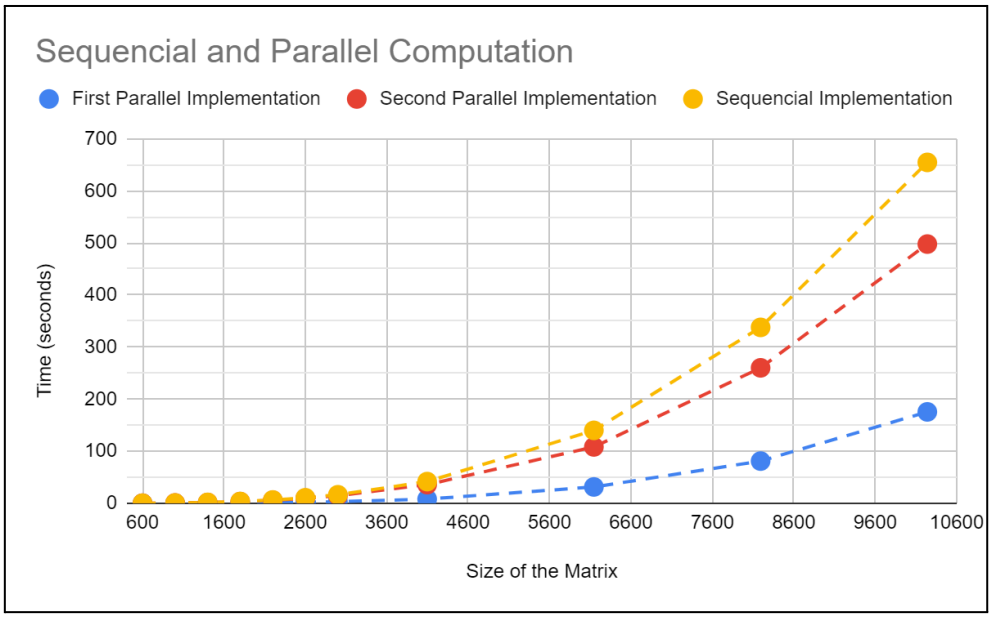
As we can see from the graph below, this algorithm is more efficient than the previous ones. This is due to the fact that we divide the same problem into several smaller ones, that is, we divide a very large matrix into several smaller ones. By dividing the matrix into smaller blocks we increase the temporal coherence of each value since the time between accesses to the same position is reduced.

We also noticed that bigger blocks tend to perform better, though in the case of matrices of 8192 x 8192 the runs with block size 128 were substantially faster than with larger blocks.



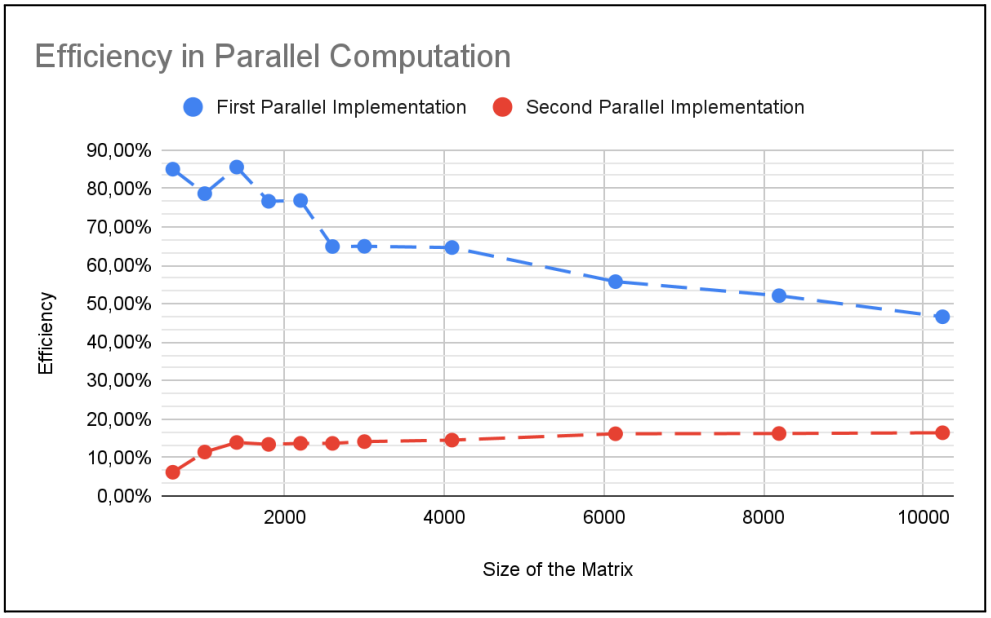
4.3. Differences in Parallel Implementations

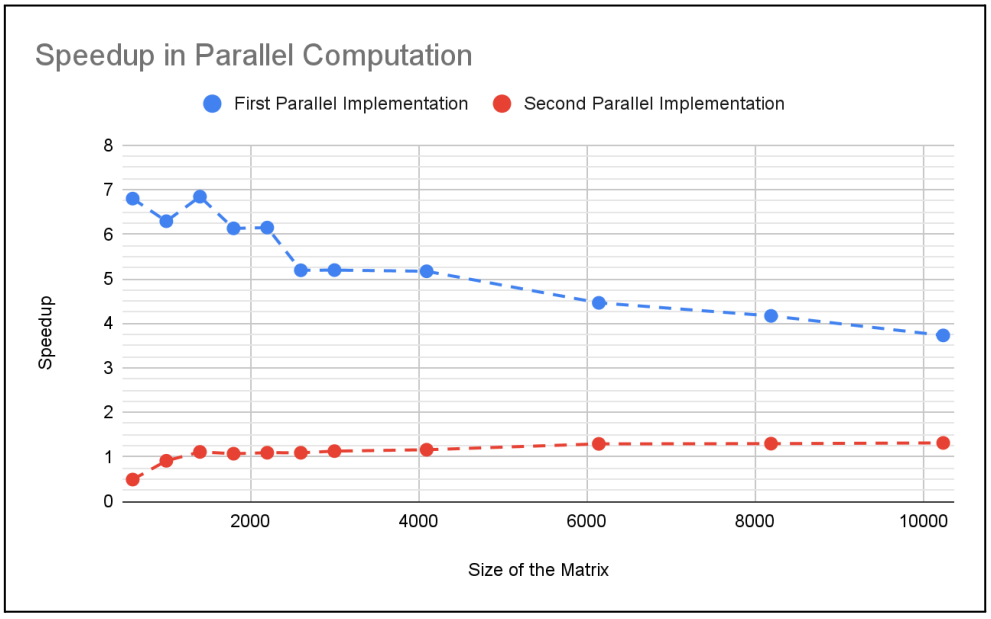
When using parallel computing, we can verify that there is a temporal improvement in the results obtained compared to those previously obtained when doing sequentially since we are using more of the processor's capabilities by leveraging multiple cores at the same time.



We started by making the first version, with the outermost loop run in parallel, which was the case where we saw better results, having a speedup between 3,5 and 7, but we also found that as we used larger matrices, efficiency decreased because the problem became more bound by the memory speed. In the second version, only the innermost loop was executed in parallel, which led to a speedup between 0,5 and 1,5, and an increase in execution time compared to the previous one, but unlike that one, its efficiency improved with an increase in matrix size.

A difference in results between the two implementations can be attributed to how parallelization is carried out. When we parallelize the outermost loop, its iterations are divided among the threads, and each thread executes a continuous block of iterations. Upon completion of these iterations, synchronization occurs between the threads. On the other hand, when we parallelize the innermost loop, all threads execute all iterations of the outer loops, while only the iterations of the inner loop are divided among them. This results in the need for synchronization between the threads after each execution of the inner loop, which can affect performance due to additional synchronization overhead.





5. Conclusion

This project was a great opportunity to verify that different implementations that perform the same kind of task can have different performances and how part of that is due to the way we access the memory.

We found that the way our program interacts with the cache can have a big impact on performance, and that to take advantage of it we must perform memory access sequentially and keep each access to the same region of memory close in time.

We also found that using parallel processing when available can also have a considerable impact on the speed of the program.

Attachments

A1 - Normal Multiplication's Algorithm Results

Size	Time	GFlops	L1 Data Cache Misses	L2 Data Cache Misses
600	0,197	2,192	244.786.999	39.332.348
1000	1,206	1,659	1.220.517.923	297.176.983
1400	3,733	1,470	3.515.366.430	1.475.410.761
1800	18,455	0,632	9.065.198.078	8.057.856.659
2200	38,736	0,550	17.651.960.391	23.669.052.623
2600	69,278	0,507	30.874.503.903	51.465.781.273
3000	115,695	0,467	50.304.911.243	95.621.943.211

Tabel 1 - C++ implementation of Normal Multiplication's algorithm

Size	Time	GFlops
600	0,234	1,846
1000	1,592	1,246
1400	5,318	1,032
1800	19,504	0,598
2200	40,748	0,523
2600	70,555	0,498
3000	117,804	0,458

Tabel 2 - Java implementation of Normal Multiplication's algorithm

A2 - Line Multiplication's Algorithm Results

Size	Time	GFlops	L1 Data Cache Misses	L2 Data Cache Misses
600	0,102	4,235	27.110.042	56.784.275
1000	0,560	3,571	125.694.485	255.911.774
1400	1,670	3,287	345.954.133	683.655.317
1800	3,434	3,400	744.447.666	1.419.374.851
2200	6,364	3,346	2.073.120.571	2.523.219.248
2600	10,678	3,202	4.412.727.956	4.103.506.971
3000	16,376	3,297	6.780.811.810	6.252.080.441
4096	41,670	3,298	17.543.697.761	16.244.374.162
6144	140,029	3,313	59.152.328.093	54.425.311.499
8192	337,829	3,255	140.151.818.769	127.285.528.681
10240	654,648	3,280	273.723.689.531	254.086.535.893

Tabel 3 - C++ implementation of Line Multiplication's algorithm

Size	Time	GFlops
600	0,183	2,360
1000	0,843	2,372
1400	2,694	2,037
1800	5,570	2,094
2200	10,295	2,068
2600	17,605	1,997
3000	26,789	2,016

Tabel 4 - Java implementation of Line Multiplication's algorithm

A3 - Block Multiplication's Algorithm Results

Size	Blocks	Time	GFlops	L1 Data Cache Misses	L2 Data Cache Misses
4096	128	36,339	3,782	9.874.057.014	33.540.547.057
	256	32,653	4,209	9.138.745.351	23.329.324.113
	512	31,294	4,392	8.767.113.992	19.835.955.975
6144	128	125,923	3,684	33.009.021.830	112.044.188.566
	256	113,779	4,077	30.766.195.434	78.422.481.106
	512	110,023	4,216	29.634.275.481	66.668.900.151
8192	128	275,374	3,993	74.368.592.969	254.693.991.399
	256	409,342	2,686	72.034.992.349	161.506.848.677
	512	345,402	3,183	70.720.933.117	144.525.902.275
10240	128	567,149	3,786	151.145.096.238	501.643.662.852
	256	480,881	4,466	141.573.377.624	356.854.514.253
	512	435,327	4,933	137.180.327.001	310.384.590.766

Tabel 5 - C++ implementation of Normal Multiplication's algorithm

A4 - First Parallel Algorithm Results

Size	Time	GFlops	L1 Data Cache Misses	L2 Data Cache Misses
600	0,015	28,800	3.391.196	7.118.572
1000	0,089	22,472	15.718.013	32.821.883
1400	0,244	20,492	43.439.415	87.355.981
1800	0,560	20,829	93.487.763	184.071.507
2200	1,035	20,576	258.623.357	335.506.533
2600	2,057	17,089	549.568.784	547.651.591
3000	3,152	17,132	845.109.581	847.485.942
4096	8,061	17,050	2.202.759.155	2.171.329.742
6144	31,400	14,772	7.420.910.498	7.210.551.881
8192	81,073	13,562	17.514.232.744	16.643.800.328
10240	175,541	12,234	34.281.996.917	31.768.125.188

Tabel 6 - First Parallel implementation using Line Multiplication algorithm

Size	Speedup	Efficiency
600	6,800	85,0%
1000	6,292	78.6%
1400	6,844	85,5%
1800	6,132	76,6%
2200	6,149	76,8%
2600	5,191	64,8%
3000	5,195	64,9%
4096	5,169	64,6%
6144	4,460	55,7%
8192	4,167	52,0%
10240	3,729	46,6%

Tabel 7 - First Parallel implementation using Line Multiplication algorithm

A5 - Second Parallel Algorithm Results

Size	Time	GFlops	L1 Data Cache Misses	L2 Data Cache Misses
600	0,207	2,087	9.249.592	37.716.956
1000	0,613	3,363	31.304.009	120.634.165
1400	1,500	3,659	73.084.815	241.585.167
1800	3,201	3,644	141.172.447	393.768.838
2200	5,812	3,664	241.539.290	618.202.538
2600	9,758	3,602	378.486.618	914.613.589
3000	14,475	3,730	563.555.260	1.291.441.125
4096	35,861	3,832	1.268.386.480	2.589.638.348
6144	108,230	4,286	4.147.609.688	8.252.086.229
8192	260,114	4,227	9.431.316.404	16.550.572.021
10240	497,789	4,314	18.499.524.700	30.650.309.124

Tabel 8 - Second Parallel implementation using Line Multiplication algorithm

Size	Speedup	Efficiency
600	0,493	6,1%
1000	0,914	11,4%
1400	1,113	13,9%
1800	1,073	13,4%
2200	1,095	13,6%
2600	1,094	13,6%
3000	1,131	14,1%
4096	1,162	14,5%
6144	1,294	16,1%
8192	1,299	16,2%
10240	1,315	16,4%

Tabel 9 - Second Parallel implementation using Line Multiplication algorithm